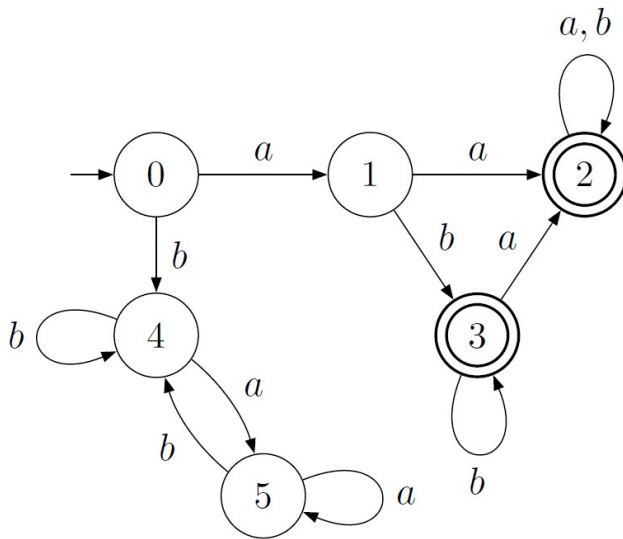




G52LAC (COMP2040)

Languages and Computation

Lecture 3: DFAs & NFAs

Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- Quick recap of previous lecture.
- Deterministic Finite Automata (DFAs).
- Nondeterministic Finite Automata (NFAs).

Outline

- Quick recap of previous lecture.
- **Deterministic Finite Automata (DFAs).**
- Nondeterministic Finite Automata (NFAs).

The Problem



See previous lecture

- An important question:

Given a language $L \subseteq \Sigma^$ and a word $w \in \Sigma^*$, can we determine if $w \in L$?*

- A possible solution:

'Construct' a machine that accepts a word w iff $w \in L$.

- Such a machine would also be a formal description of L .

Informal DFA rules

- A machine has a finite number of states.
- A machine is in exactly one state at a time.
- Input is read in one symbol at a time.
- State changes happen according to the current state and the current symbol.

Formal Definition of a DFA (1)

A DFA D is a 5-tuple, $D = (Q, \Sigma, \delta, q_0, F)$, where:

- Q is a finite set of states
- Σ is an alphabet
- $\delta \in (Q \times \Sigma \rightarrow Q)$ is a transition function
- $q_0 \in Q$ is the initial state
- $F \subseteq Q$ is a **set** of accepting (or final) states

Formal Definition of a DFA (2)

As an example consider the following automaton

$$D = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta_D, q_0, \{q_2\})$$

where

$$\delta_D = \{((q_0, 0), q_1), ((q_0, 1), q_0), ((q_1, 0), q_1), ((q_1, 1), q_2), ((q_2, 0), q_2), ((q_2, 1), q_2)\}$$

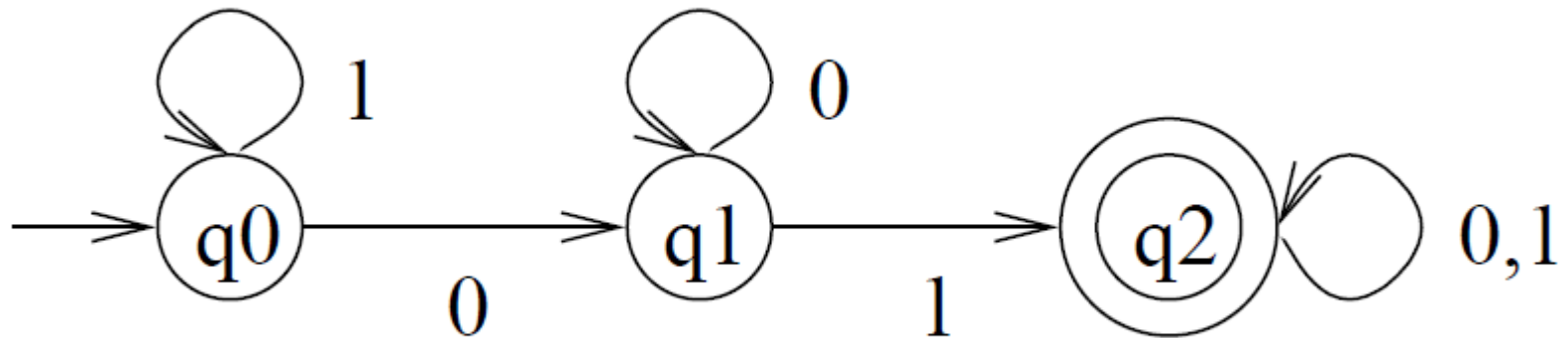
The DFA may be more conveniently represented by a transition table:

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_2	q_2

Transition table

Transition Diagrams

	0	1
$\rightarrow q_0$	q_1	q_0
q_1	q_1	q_2
$*q_2$	q_2	q_2



The language of a DFA (1)

To each DFA A we associate a language $L(A) \subseteq \Sigma^*$. To see whether a word $w \in L(A)$ we put a marker in the initial state and when reading a symbol forward the marker along the edge marked with this symbol. When we are in an accepting state at the end of the word then $w \in L(A)$, otherwise $w \notin L(A)$.

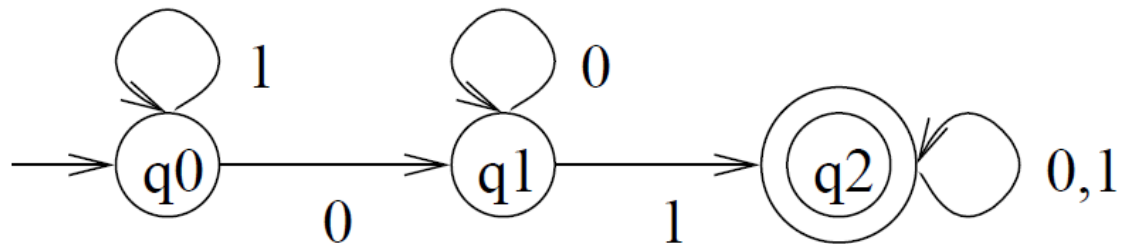
In the example above we have that $0 \notin L(D)$, $101 \in L(D)$ and $110 \notin L(D)$.

Try the following words:



- a. 1110
- b. 000
- c. 101
- d. 1101
- e. 1000
- f. 10010011

So now you can see it:
a **finite** description representing
an **infinite** language and an easy
method for checking language
membership.



The language of a DFA (2)

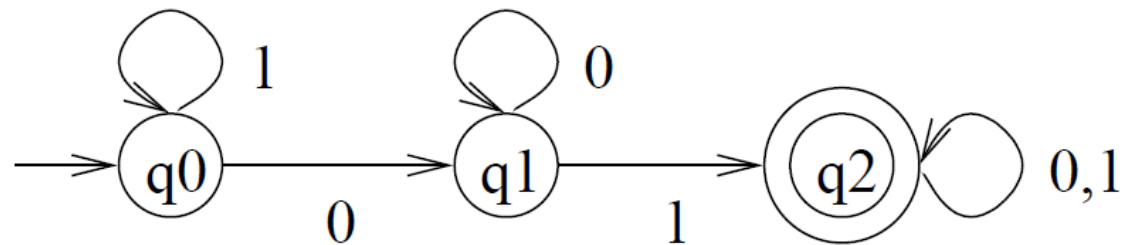
To each DFA A we associate a language $L(A) \subseteq \Sigma^*$. To see whether a word $w \in L(A)$ we put a marker in the initial state and when reading a symbol forward the marker along the edge marked with this symbol. When we are in an accepting state at the end of the word then $w \in L(A)$, otherwise $w \notin L(A)$.

In the example above we have that $0 \notin L(D)$, $101 \in L(D)$ and $110 \notin L(D)$. Indeed, we have

$$L(D) = \{w \mid w \text{ contains the substring } 01\}$$

Try the following words:

- a. 1110
- b. 000
- c. 101
- d. 1101
- e. 1000
- f. 10010011



The Extended Transition Function (1)

The transition function can be extended to operate on words (rather than symbols):

$$\hat{\delta} \in (Q \times \Sigma^* \rightarrow Q)$$

$$\hat{\delta}(q, \varepsilon) = q \quad (1)$$

$$\hat{\delta}(q, xw) = \hat{\delta}(\delta(q, x), w) \quad (2)$$

where

$$q \in Q$$

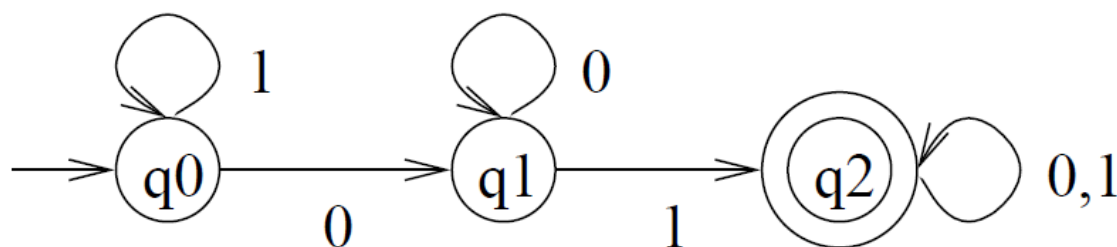
$$x \in \Sigma$$

$$w \in \Sigma^*$$

Intuitively, if we start in state q , read an input word w , and end up in state q' , then $\hat{\delta}(q, w) = q'$

The Extended Transition Function (2)

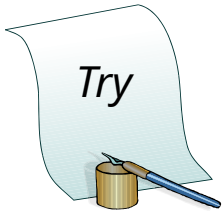
$$\begin{aligned}\hat{\delta}_D(q_0, 101) &= \hat{\delta}_D(\delta_D(q_0, 1), 01) && \text{by (2)} \\ &= \hat{\delta}_D(q_0, 01) && \text{because } \delta_D(q_0, 1) = q_0 \\ &= \hat{\delta}_D(\delta_D(q_0, 0), 1) && \text{by (2)} \\ &= \hat{\delta}_D(q_1, 1) && \text{because } \delta_D(q_0, 0) = q_1 \\ &= \hat{\delta}_D(\delta_D(q_1, 1), \epsilon) && \text{by (2)} \\ &= \hat{\delta}_D(q_2, \epsilon) && \text{because } \delta_D(q_1, 1) = q_2 \\ &= q_2 && \text{by (1)}\end{aligned}$$



The language of a DFA (3)

Given a DFA $D = (Q, \Sigma, \delta, q_0, F)$, the language of D is defined as:

$$L(D) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$



- Create a DFA with at least 3 states.
- Questions?
- Generate 6 words from your DFA.
- Give 6 words that are rejected by your DFA.
- Questions?

$$\hat{\delta} \in (Q \times \Sigma^* \rightarrow Q)$$

$$\hat{\delta}(q, \varepsilon) = q \quad (1)$$

$$\hat{\delta}(q, xw) = \hat{\delta}(\delta(q, x), w) \quad (2)$$

where

$$q \in Q$$

$$x \in \Sigma$$

$$w \in \Sigma^*$$

Reading Sources (DFA)

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 45–52
- G52MAL Lecture Notes, pages 7–8

Outline

- Quick recap of previous lecture.
- Deterministic Finite Automata (DFAs).
- Nondeterministic Finite Automata (NFAs).

DFA Recap

- DFA $D = (Q, \Sigma, \delta, q_0, F)$, where:
 - Q : Finite set of states
 - Σ : Alphabet
 - $\delta \in (Q \times \Sigma \rightarrow Q)$: Transition function
 - $q_0 \in Q$: Initial state
 - $F \subseteq Q$: Set of accepting states

- Extended Transition Function:

$$\hat{\delta} \in (Q \times \Sigma^* \rightarrow Q)$$

$$\hat{\delta}(q, \varepsilon) = q$$

$$\hat{\delta}(q, xw) = \hat{\delta}(\delta(q, x), w)$$

- Language of a DFA:

$$L(D) = \{w \mid \hat{\delta}(q_0, w) \in F\}$$

Informal NFA rules

- NFAs are mostly the same as DFAs, except:
 - An NFA can “make choices”.
 - A word is accepted if choices can be made such that the machine ends in a final state.
- Another way of looking at this is that:
 - An NFA can be in multiple states at the same time.
 - A word is accepted if any of the ending states are final.

Formal definition of an NFA

A NFA N is a 5-tuple, $N = (Q, \Sigma, \delta, S, F)$, where:

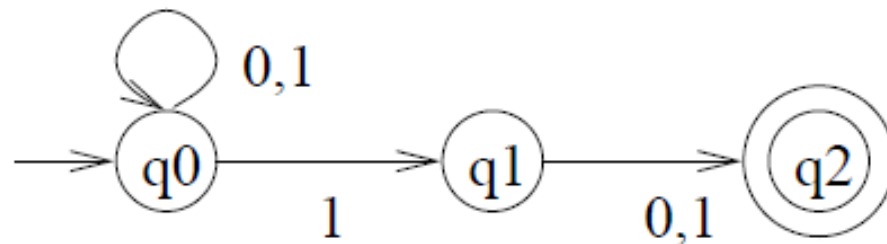
- Q is a finite set of states
- Σ is an alphabet
- $\delta \in (Q \times \Sigma \rightarrow \mathcal{P}(Q))$ is a transition function
- $S \subseteq Q$ is a set of initial states
- $F \subseteq Q$ is a set of accepting (or final) states

Example NFA - transition diagram

Here is an example of an NFA C that accepts all words over $\Sigma = \{0, 1\}$ s.t. the symbol before the last is 1.

Example NFA - transition diagram

Here is an example of an NFA C that accepts all words over $\Sigma = \{0, 1\}$ s.t. the symbol before the last is 1.



Main differences with DFAs

- The main differences with DFAs are that NFAs can have multiple start states instead of a single one and the type of the transition function. As an example, we have that:

$$C = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta_C, \{q_0\}, \{q_2\})$$

where δ_C is given by

δ_C	0	1
$\rightarrow q_0$	$\{q_0\}$	$\{q_0, q_1\}$
q_1	$\{q_2\}$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

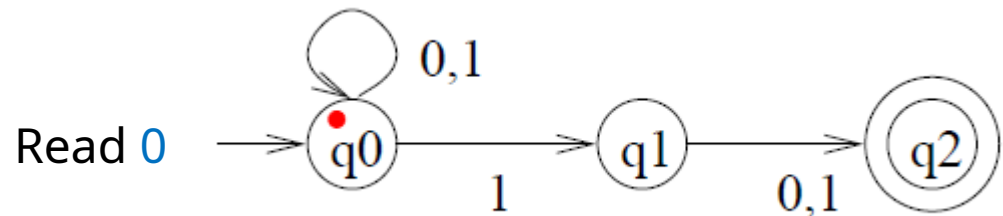
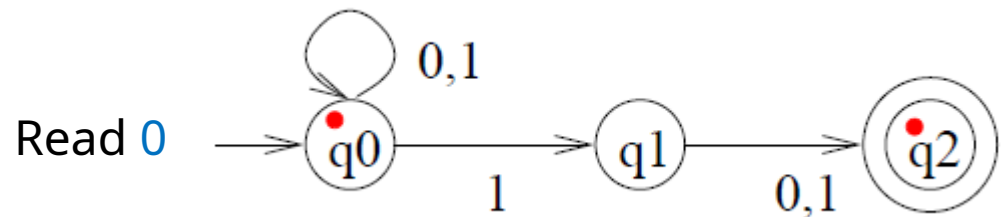
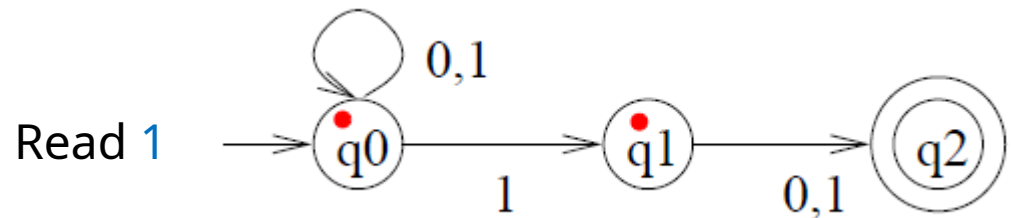
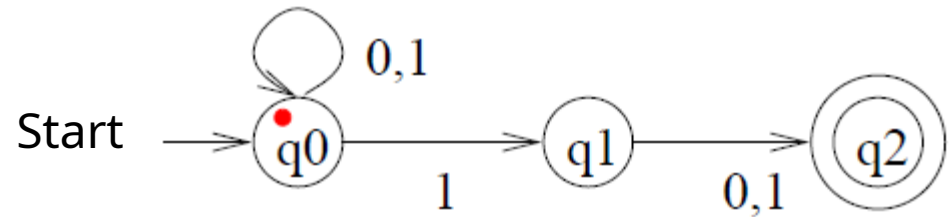
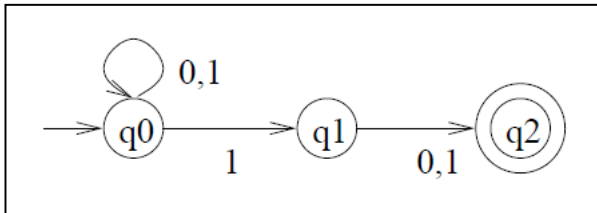
The language accepted by an NFA (1)

- To see whether a word $w \in \Sigma^*$ is accepted by an NFA ($w \in L(A)$) we may have to use **several markers**.
- Initially we put one **marker** on **each initial state**.
- Then each time we read a symbol we look at all the markers: we remove the old marker and put markers at **all the states that are reachable** via an arrow marked with the current input symbol (this may include states that were previously marked).
- Thus, we may have to use several markers, but it may also happen that **all markers disappear** (if no appropriate arrows exist). In this case the **word is not accepted**.
- If at the end of the word **any of the final states has a marker** on it then the **word is accepted**.

The language accepted by an NFA (2)

- Consider the word **100** (which is not accepted by C).

NFA C



The language accepted by an NFA (3)

To specify the extended transition function for NFAs we use an generalisation of the union operation $\cup$ on sets. We define $\bigcup$ to be the union of a (finite) set of sets:

$$\bigcup\{A_1, A_2, \dots, A_n\} = A_1 \cup A_2 \cup \dots \cup A_n$$

In the special cases of the empty sets of sets and a one element set of sets we define:

$$\bigcup \emptyset = \emptyset \quad \bigcup \{A\} = A$$

As an example

$$\bigcup\{\{1\}, \{2, 3\}, \{1, 3\}\} = \{1\} \cup \{2, 3\} \cup \{1, 3\} = \{1, 2, 3\}$$

The language accepted by an NFA (4)

We define $\hat{\delta} \in \mathcal{P}(Q) \times \Sigma^* \rightarrow \mathcal{P}(Q)$ with the intention that $\hat{\delta}(S, w)$ is the set of states that is marked after having read w starting with the initial markers given by S .

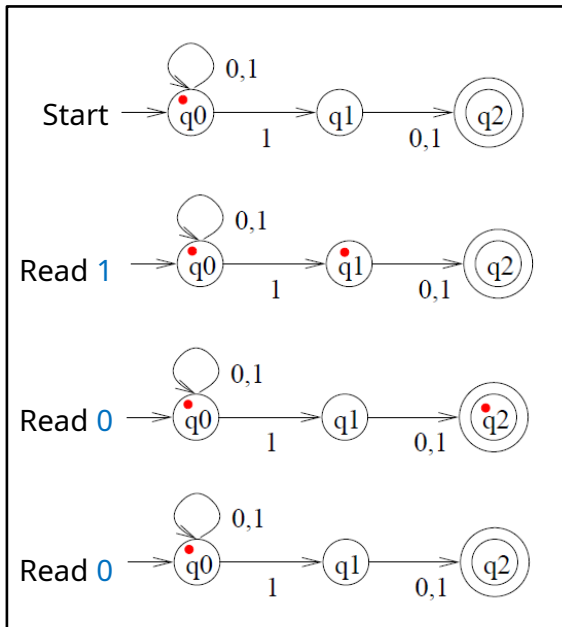
$$\hat{\delta}(P, \epsilon) = P \tag{3}$$

$$\hat{\delta}(P, xw) = \hat{\delta}\left(\bigcup\{\delta(q, x) \mid q \in P\}, w\right) \tag{4}$$

The language accepted by an NFA (5)

As an example we calculate $\hat{\delta}_C(q_0, 100)$ which is $\{q_0\}$ as we already know from playing with markers.

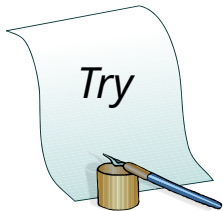
$$\begin{aligned}
 \hat{\delta}_C(\{q_0\}, 100) &= \hat{\delta}_C(\bigcup\{\delta_C(q, 1) \mid q \in \{q_0\}\}, 00) && \text{by (4)} \\
 &= \hat{\delta}_C(\delta_C(q_0, 1), 00) \\
 &= \hat{\delta}_C(\{q_0, q_1\}, 00) \\
 &= \hat{\delta}_C(\bigcup\{\delta_C(q, 0) \mid q \in \{q_0, q_1\}\}, 0) && \text{by (4)} \\
 &= \hat{\delta}_C(\delta_C(q_0, 0) \cup \delta_C(q_1, 0), 0) \\
 &= \hat{\delta}_C(\{q_0\} \cup \{q_2\}, 0) \\
 &= \hat{\delta}_C(\{q_0, q_2\}, 0) \\
 &= \hat{\delta}_C(\bigcup\{\delta_C(q, 0) \mid q \in \{q_0, q_2\}\}, \epsilon) && \text{by (4)} \\
 &= \hat{\delta}_C(\delta_C(q_0, 0) \cup \delta_C(q_2, 0), \epsilon) \\
 &= \hat{\delta}_C(\{q_0\} \cup \emptyset, \epsilon) \\
 &= \{q_0\} && \text{by (3)}
 \end{aligned}$$



The language accepted by an NFA (6)

Given an NFA $N = (Q, \Sigma, \delta, S, F)$, the language of N is defined as:

$$L(N) = \{w \mid (\hat{\delta}(S, w) \cap F) \neq \emptyset\}$$



- Create an NFA with at least 3 states and an alphabet with at least 3 elements.
- Questions?
- Generate 6 words from your NFA.
- Give 6 words that are rejected by your NFA.
- Questions?

Reading Sources (NFA)

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 55–59
- G52MAL Lecture Notes, pages 9–11

Acknowledgements

- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.